Viva Preparation - Member 3 - Booking Service

Owned Component

Owned microservice:

- `booking-service`

Main files to show:

- `services/booking-service/src/app.js`
- `services/booking-service/src/controllers/bookingController.js`
- `services/booking-service/src/services/eventService.js`
- `services/booking-service/src/services/authService.js`
- `services/booking-service/src/services/notificationService.js`
- `services/booking-service/src/models/Booking.js`
- `services/booking-service/Dockerfile`
- `sonar/booking-service.properties`

What To Say

I implemented the booking orchestration service. This is the service that coordinates the booking workflow across multiple microservices.

The booking service handles booking requests, pending approvals, confirmation, rejection, cancellation, and booking management views for organizers and admins.

This is the most integration-heavy service because it communicates with:

- `event-service`
- `notification-service`
- `auth-service`

Endpoints To Explain

- `POST /api/bookings` - creates a booking request.
- `GET /api/bookings/user/:userId` - lists bookings for a customer.
- `GET /api/bookings/organizer/queue` - lists pending approvals.
- `GET /api/bookings/management` - lists managed bookings for organizer/admin.
- `GET /api/bookings/:id` - gets one booking.
- `PATCH /api/bookings/:id/confirm` - confirms a pending booking.
- `PATCH /api/bookings/:id/reject` - rejects a pending booking.
- `PATCH /api/bookings/:id/cancel` - cancels a booking.

Through the gateway these become:

- `/bookings`
- `/bookings/user/:userId`
- `/bookings/organizer/queue`

- `/bookings/management`

Integration Points

Booking Service To Event Service

When a customer requests a booking, the booking service calls event-service to:

- fetch event details
- reserve seats

When a booking is rejected or cancelled, it calls event-service to:

- release seats

Booking Service To Notification Service

After booking actions, the booking service sends notification creation requests to notification-service.

Examples:

- customer gets pending notification
- organizer gets booking request notification
- admin gets booking request notification
- customer gets confirmation/rejection notification

Booking Service To Auth Service

The booking service calls auth-service to find admin users so admin notifications can be created.

Demo Steps

This is the strongest viva demo flow.

1. Log in as organizer.
2. Create an event.
3. Log in as customer.
4. Request a booking.
5. Explain that booking-service calls event-service to reserve seats.
6. Explain that booking-service calls notification-service to create alerts.
7. Log in as organizer/admin.
8. Open approval queue.
9. Confirm or reject the booking.
10. Show customer notification update.

Booking State Flow

Possible statuses:

- `pending`
- `confirmed`

- `rejected`
- `cancelled`

Normal flow:

1. Customer creates request.
2. Booking becomes `pending`.
3. Organizer/admin confirms or rejects.
4. Customer can cancel if applicable.

Security Points

- Booking routes are protected by JWT.
- Customers can only view their own bookings.
- Organizers can manage bookings related to their events.
- Admins can oversee global bookings.
- Internal service calls use service URLs from environment variables.
- In Azure, backend services are internal and accessed through gateway/service-to-service communication.

DevOps And Cloud Points

- The service has its own Dockerfile.
- The Docker image is published to GHCR.
- The service is deployed to Azure Container Apps.
- The service has a separate SonarCloud project:
  - `EventHub Booking Service`
- The GitHub Actions workflow scans this service using:
  - `sonar/booking-service.properties`

Likely Viva Questions

What is the purpose of the booking service?

It manages the booking lifecycle: creating requests, storing pending bookings, approval by organizer/admin, rejection, confirmation, cancellation, and booking management.

Why is this service an orchestration service?

Because it coordinates multiple services. It calls event-service for seats, notification-service for alerts, and auth-service to find admin users.

What is your main inter-service communication demo?

The booking request flow. A customer booking triggers calls from booking-service to event-service and notification-service.

What happens when a booking is rejected?

The booking status becomes `rejected`, seats are released through event-service, and the customer receives a notification through notification-service.

What happens when a booking is confirmed?

The booking status becomes `confirmed`, and the customer receives a confirmation notification.

Why did you use pending status first?

Because the system supports an approval workflow. Customers request bookings, and organizers/admins approve or reject them.

How is data consistency handled between booking and event seats?

The booking service reserves seats through the event service during booking creation. If the booking is rejected or cancelled, it calls the event service to release seats.

What would you improve in production?

I would add stronger transaction handling or eventual consistency mechanisms, retries, idempotency keys, and possibly a message queue for notification workflows.

Common Group Questions

What is the application?

EventHub is a microservice-based event booking platform. Customers can browse events and request bookings, organizers can manage events and approvals, and admins can oversee activity.

What are the four microservices?

- Auth Service
- Event Service
- Booking Service
- Notification Service

What is the API gateway used for?

The gateway is the public backend entry point. The frontend calls the gateway, and the gateway routes requests to the correct internal service.

What is the main integration flow?

The strongest flow is booking:

1. Customer requests a booking.
2. Booking service calls event service to check and reserve seats.
3. Booking service stores a pending booking.
4. Booking service calls notification service to create alerts.
5. Booking service calls auth service when admin user data is needed.

What DevOps practices are implemented?

- GitHub repository
- GitHub Actions CI
- Docker containerization
- GHCR container image publishing
- Azure Container Apps deployment
- SonarCloud DevSecOps scanning

What security practices are implemented?

- JWT authentication
- Role-based access
- Internal microservice ingress in Azure
- Environment variables and GitHub secrets
- SonarCloud static analysis
- Express Helmet middleware

Quick Emergency Checks

Gateway health:

```
curl https://gateway.mangocoast-efcefaea.southeastasia.azurecontainerapps.io
```

Booking logs:

```
az containerapp logs show --name booking-service --resource-group eventhub-rg --
tail 100
```

Gateway logs:

```
az containerapp logs show --name gateway --resource-group eventhub-rg --tail 100
```